

GreenWave EMS

Emergency Management System – Architettura IoT ibrida per la gestione intelligente del traffico di emergenza

Corso: Internet of Things (IoT) 2025/2026

Componenti del gruppo: Giorgio Innocenti, Alessio Scarpulla

Tecnologie principali: Node.js, Node-RED, MQTT, ESP32, Wokwi, C++, Grafana, Worldmap, Ollama e Large Language Model

Tipologia di sistema: Hybrid IoT / Smart City / Emergency Vehicle Management

1. Introduzione

Il progetto **GreenWave EMS (Emergency Management System)** nasce con l'obiettivo di progettare e realizzare un sistema IoT in grado di migliorare la gestione del traffico urbano durante il transito di un veicolo di emergenza, in particolare un'ambulanza in stato di emergenza.

L'idea alla base del progetto è relativamente semplice: quando un'ambulanza deve attraversare un'intersezione stradale durante un'emergenza, il normale funzionamento del semaforo può costituire un ostacolo significativo. Il sistema sviluppato permette quindi di trasformare l'intersezione in un'infrastruttura **reattiva**, capace di riconoscere l'avvicinamento del mezzo di emergenza e predisporre automaticamente una condizione favorevole al suo attraversamento.

Il progetto non si limita tuttavia alla semplice accensione del semaforo verde.

L'architettura realizzata integra infatti:

- un simulatore dell'ambulanza sviluppato in Node.js;
- un'intersezione intelligente basata su ESP32 e simulata tramite Wokwi;
- un sensore a ultrasuoni HC-SR04;
- tre LED per la rappresentazione del semaforo;
- un servomotore per simulare la barriera di sicurezza;
- una logica di sicurezza locale sull'ESP32;
- un broker MQTT per la comunicazione tra dispositivi;
- Node-RED come piattaforma centrale di elaborazione e coordinamento;
- un sistema di monitoraggio operativo con visualizzazione geografica;
- Grafana per l'analisi e la visualizzazione storica dei dati;
- un sistema di logging e generazione di file CSV;
- la possibilità per l'utente di scaricare il CSV direttamente sul proprio dispositivo;
- un'integrazione con un Large Language Model locale tramite Ollama;
- una pipeline automatica per la generazione di un report operativo al termine dell'evento di emergenza.

Il risultato è quindi un sistema IoT distribuito nel quale diversi componenti cooperano attraverso protocolli e interfacce differenti, mantenendo una chiara separazione tra acquisizione dei dati, comunicazione, elaborazione, attuazione e visualizzazione.

2. Idea iniziale e obiettivo applicativo

L'idea iniziale del progetto nasce da un problema concreto delle Smart Cities: la gestione del traffico durante il passaggio dei veicoli di emergenza.

In una situazione reale, un'ambulanza in emergenza può incontrare diversi ostacoli:

- semafori rossi;
- traffico intenso;
- veicoli fermi all'incrocio;
- attraversamenti pedonali;
- tempi di attesa dovuti alla normale temporizzazione dei semafori.

L'obiettivo di GreenWave EMS è quindi introdurre un meccanismo di **pre-emption del traffico**, nel quale il normale funzionamento dell'intersezione viene temporaneamente modificato in risposta alla presenza di un veicolo di emergenza.

Il sistema segue concettualmente questa sequenza:

```
Ambulanza in emergenza
↓
Invio posizione tramite MQTT
↓
Node-RED riceve la telemetria
↓
Calcolo della distanza dall'intersezione
↓
Verifica della soglia di intervento
↓
Attivazione della Green Wave
↓
Invio comando MQTT all'intersezione
↓
ESP32 riceve il comando
↓
Verifica locale delle condizioni di sicurezza
↓
Attuazione del semaforo/barriera
↓
Attraversamento dell'ambulanza
↓
Conclusione dell'emergenza
↓
Ripristino dell'intersezione
↓
Registrazione dell'evento
↓
Analisi tramite LLM
```

↓
Generazione del report operativo

La caratteristica fondamentale dell'architettura è che **la decisione di orchestrazione è centralizzata**, mentre la sicurezza dell'attuazione viene mantenuta sul dispositivo Edge.

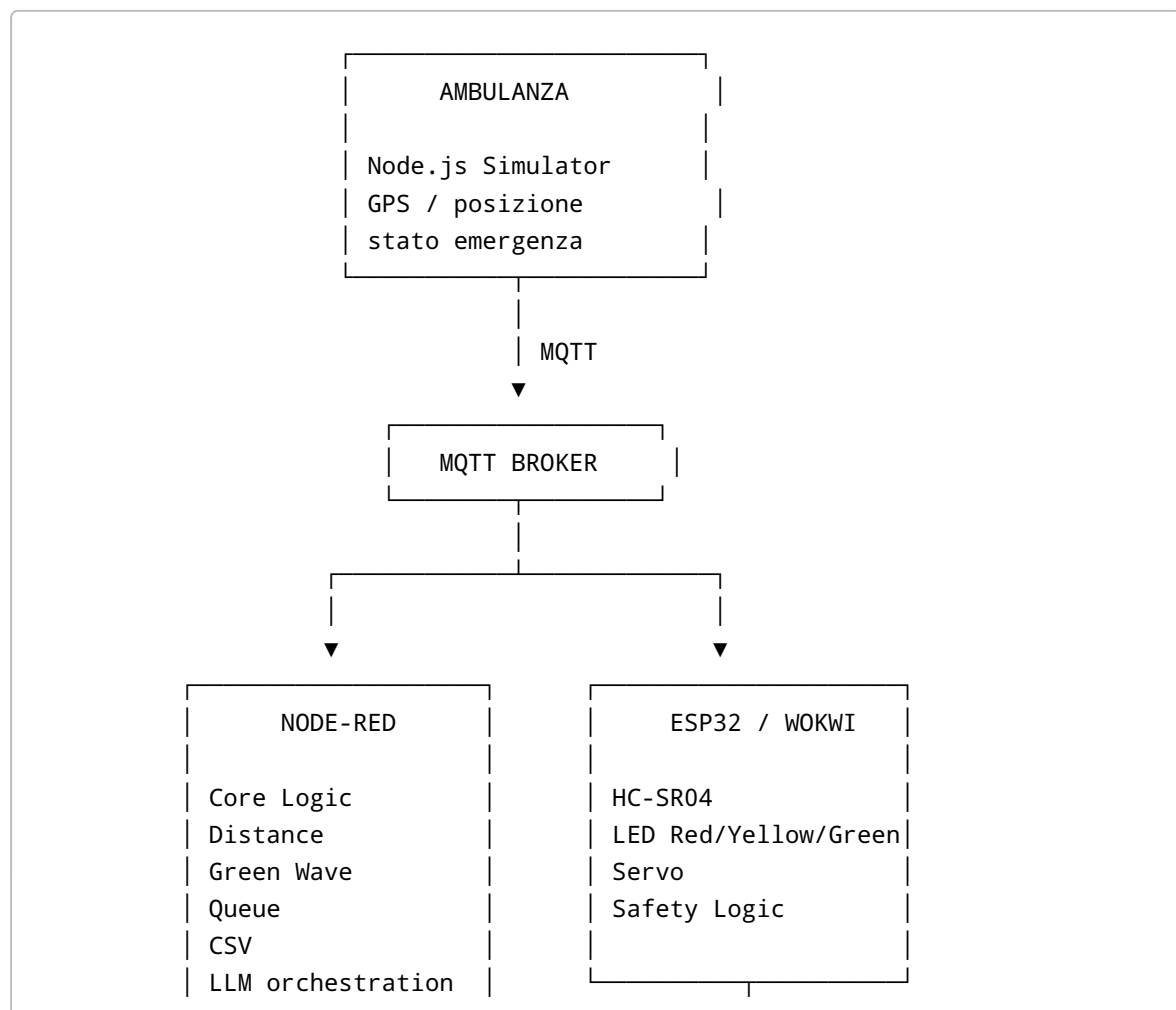
3. Architettura generale

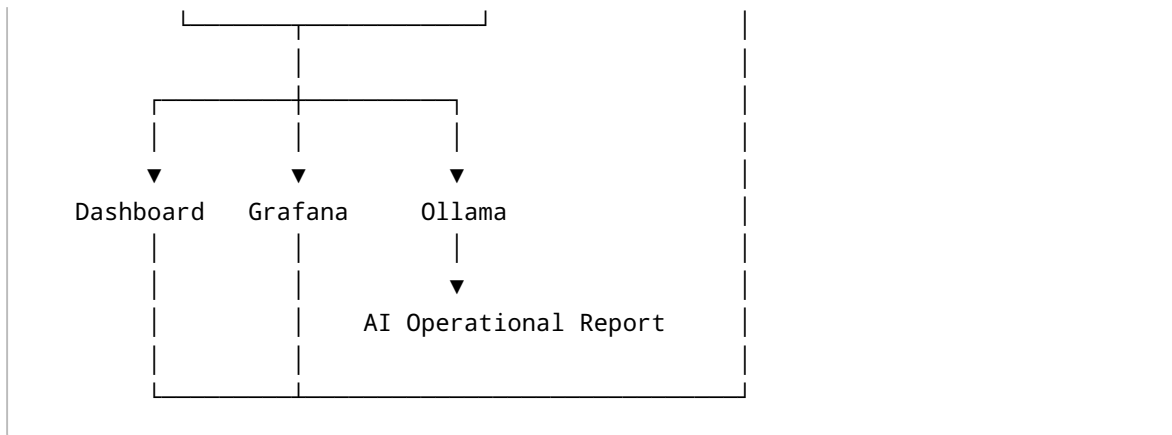
Il progetto è stato sviluppato secondo una struttura IoT distribuita riconducibile alle principali componenti di un'architettura IoT:

1. **Perception / Edge Layer**
2. **Transport Layer**
3. **Processing Layer**
4. **Application / Visualization Layer**

A queste componenti è stata successivamente aggiunta una componente di **Artificial Intelligence**, utilizzata per l'analisi post-evento.

L'architettura complessiva può essere rappresentata come segue:





Questa struttura permette di mantenere separati i compiti dei singoli componenti.

4. I due Edge Device

Uno degli elementi fondamentali del progetto è la presenza di due dispositivi Edge distinti.

4.1 Edge Device 1 – Ambulanza

Il primo dispositivo Edge rappresenta il veicolo di emergenza.

Non essendo necessario disporre di un vero GPS o di un'ambulanza fisica, il comportamento del mezzo viene simulato tramite un'applicazione **Node.js**.

Il simulatore produce periodicamente dati relativi alla posizione del veicolo e allo stato dell'emergenza.

La posizione viene rappresentata mediante coordinate che simulano l'avvicinamento progressivo all'intersezione.

Il simulatore agisce quindi come un dispositivo IoT che acquisisce dati e li pubblica sulla rete.

Il comportamento concettuale è:

```
Generazione posizione
↓
Aggiornamento coordinate
↓
Creazione JSON
↓
Pubblicazione MQTT
```

Un esempio concettuale di messaggio può essere:

```
{
  "latitude": 44.40,
  "longitude": 8.93,
  "siren_active": true
}
```

Il formato effettivamente utilizzato dal progetto può naturalmente contenere ulteriori campi, ma il principio è quello di trasformare la telemetria del veicolo in un messaggio strutturato trasmissibile tramite MQTT.

5. Edge Device 2 – Smart Intersection

Il secondo Edge Device rappresenta l'intersezione intelligente.

È basato su un **ESP32** e viene sviluppato e testato tramite **Wokwi**, consentendo di simulare l'hardware senza dipendere da componenti fisici.

L'intersezione contiene diversi elementi:

- ESP32;
- LED rosso;
- LED giallo;
- LED verde;
- sensore HC-SR04;
- servomotore per la simulazione della barriera.

Il dispositivo non viene utilizzato semplicemente come un attuatore remoto.

L'ESP32 possiede infatti una propria logica locale, particolarmente importante per la gestione della sicurezza.

6. Sensore HC-SR04

Il sensore a ultrasuoni **HC-SR04** viene utilizzato per rilevare la presenza di un ostacolo nell'area controllata dall'intersezione.

Il principio di funzionamento del sensore è basato sulla misura del tempo impiegato da un impulso ultrasonico per raggiungere un ostacolo e tornare al sensore.

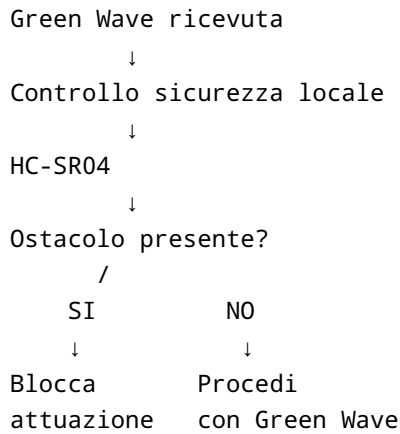
Il microcontrollore può quindi ricavare la distanza dell'ostacolo.

Nel contesto GreenWave EMS, questo dato viene utilizzato come condizione di sicurezza.

L'obiettivo non è semplicemente:

Green Wave ricevuta → semaforo verde

ma:



Questo introduce un importante principio di **Edge Computing**.

La decisione finale relativa alla sicurezza non dipende esclusivamente dal backend.

7. Attuatori dell'intersezione

L'intersezione utilizza diversi attuatori.

7.1 LED

I tre LED rappresentano il semaforo:

- rosso;
- giallo;
- verde.

Essi consentono di osservare direttamente il comportamento dell'intersezione all'interno della simulazione Wokwi.

7.2 Servomotore

Il servomotore rappresenta una barriera fisica, utilizzata nel progetto come simulazione di un sistema di blocco/protezione dell'area di attraversamento.

Il servo viene controllato dall'ESP32 tramite un segnale PWM.

In questo modo il sistema può simulare il passaggio da una condizione normale a una condizione di apertura/blocco associata alla gestione dell'emergenza.

8. Safety Logic

La Safety Logic costituisce uno degli aspetti architetturali più importanti del progetto.

Node-RED possiede il compito di coordinare il sistema.

Quando l'ambulanza si avvicina sufficientemente all'intersezione, Node-RED può inviare un comando di Green Wave tramite MQTT.

Tuttavia, il comando non deve bypassare i controlli locali.

L'ESP32 riceve quindi il comando e verifica autonomamente la situazione rilevata dal sensore.

Questa scelta introduce una separazione tra:

orchestrazione

e

sicurezza locale.

Il backend può dire:

"È necessario preparare l'intersezione per il passaggio dell'ambulanza."

L'ESP32 può invece stabilire:

"Posso procedere con l'attuazione solo se le condizioni locali sono sicure."

Questo modello è particolarmente appropriato per un sistema IoT con componenti fisici, perché evita che un problema della rete o del backend comporti automaticamente un comportamento non sicuro dell'attuatore.

9. MQTT e comunicazione tra dispositivi

Il protocollo di comunicazione scelto è **MQTT (Message Queuing Telemetry Transport)**.

La scelta di MQTT è coerente con le caratteristiche del progetto perché il protocollo utilizza un modello publish/subscribe e permette di disaccoppiare produttori e consumatori dei dati.

Il sistema utilizza un broker MQTT come intermediario.

Invece di avere una comunicazione diretta:

Ambulanza → Node-RED

si utilizza:

Ambulanza → MQTT Broker → Node-RED

e analogamente:

Node-RED → MQTT Broker → ESP32

Questo rende l'architettura più modulare.

10. Topic MQTT principali

Nel progetto sono stati utilizzati topic distinti per separare le diverse categorie di informazioni.

Tra quelli visibili nell'implementazione rientrano:

Telemetria dell'ambulanza

city/ems/ambulance1/gps

Questo topic trasporta i dati relativi alla posizione dell'ambulanza.

Comandi all'intersezione

city/ems/trafficlight1/cmd

Questo topic viene utilizzato dal backend per inviare comandi all'intersezione, inclusa la gestione della Green Wave.

Stato dell'intersezione

city/ems/trafficlight1/status

Permette al backend di ricevere informazioni sullo stato del semaforo e sulle condizioni dell'intersezione.

Coda del traffico

```
city/ems/trafficlight1/queue
```

Questo topic permette di monitorare la situazione del traffico e della coda presso l'intersezione.

La separazione dei topic consente di evitare di concentrare tutte le informazioni in un unico canale.

11. Node-RED come centro di elaborazione

Node-RED costituisce il cuore del sistema di processing.

Nel progetto non viene utilizzato solamente come strumento grafico per collegare nodi, ma come vero e proprio **Stream Processor**.

Riceve dati MQTT, li elabora, calcola informazioni derivate, decide quando attivare la Green Wave e inoltra i risultati verso altri componenti.

Nel flow sono presenti diversi blocchi funzionali.

Tra i principali:

- Core logic (green wave)
- function mapper
- Init Marker Incrocio
- function arrival
- Aggiorna colore semaforo
- Check pedone
- Estrazione colore
- Formattazione csv
- Memoria Coda
- Generazione Prompt LLM
- Estrazione Output IA

La suddivisione in Function Node permette di separare le diverse responsabilità logiche.

12. Ricezione della telemetria

Il flusso principale parte dalla ricezione del messaggio MQTT relativo all'ambulanza.

Il topic:

```
city/ems/ambulance1/gps
```

alimenta la logica centrale.

I dati ricevuti vengono quindi utilizzati per:

- aggiornare la posizione;
- calcolare la distanza dall'incrocio;
- visualizzare l'ambulanza;
- determinare l'avvicinamento alla soglia di intervento;
- alimentare il sistema di logging;
- contribuire all'analisi dell'evento.

13. Calcolo della distanza

Una delle funzioni fondamentali del backend è il calcolo della distanza dell'ambulanza dall'intersezione.

La distanza rappresenta uno dei principali parametri decisionali del sistema.

Il comportamento concettuale è:

```
Posizione ambulanza
      +
Posizione incrocio
      ↓
Calcolo distanza
      ↓
Confronto con soglia
```

Quando l'ambulanza è sufficientemente lontana, il sistema mantiene il normale funzionamento.

Quando la distanza scende sotto la soglia configurata, il sistema può iniziare la procedura di pre-emption.

Questo permette di evitare un'attivazione prematura della Green Wave.

14. Green Wave

La Green Wave rappresenta il principale comportamento automatico del progetto.

Il concetto consiste nel preparare l'intersezione per il passaggio dell'ambulanza.

Il flusso può essere rappresentato come:

```
Ambulanza
      ↓
```

```
Posizione
  ↓
Distanza dall'incrocio
  ↓
Soglia raggiunta
  ↓
Core Logic
  ↓
Green Wave
  ↓
MQTT command
  ↓
ESP32
```

La Green Wave non viene quindi attivata manualmente dall'operatore durante il normale funzionamento.

È una conseguenza della telemetria e della logica di elaborazione.

15. Reset dell'intersezione

Oltre all'attivazione della Green Wave, il progetto implementa anche il ritorno alla situazione normale.

Nel flow è presente un meccanismo di reset dell'incrocio.

È infatti necessario evitare che l'intersezione rimanga indefinitamente nello stato di emergenza dopo il passaggio del veicolo.

Il comportamento generale è:

```
Normal operation
  ↓
Emergency detected
  ↓
Green Wave
  ↓
Ambulance crossing
  ↓
Emergency completed
  ↓
Reset intersection
  ↓
Normal operation
```

Questa componente completa il ciclo di vita dell'evento.

16. Monitoraggio del traffico

Una funzionalità sviluppata durante il progetto riguarda anche la gestione della coda di traffico.

Il sistema riceve dati dal topic:

```
city/ems/trafficlight1/queue
```

e li elabora attraverso Node-RED.

Nel flow sono presenti componenti denominati:

- Coda traffico
- Memoria Coda

e un'ulteriore elaborazione verso un endpoint HTTP relativo alla coda.

Questo permette di mantenere uno storico dell'andamento del traffico.

La funzionalità è particolarmente interessante perché consente di valutare non solo l'efficacia della Green Wave per l'ambulanza, ma anche il suo impatto sul traffico ordinario.

17. Dashboard operativa

Il progetto comprende una dashboard operativa dedicata al monitoraggio in tempo reale.

La dashboard permette all'operatore di osservare informazioni quali:

- posizione dell'ambulanza;
- movimento dell'ambulanza;
- distanza dall'intersezione;
- stato del semaforo;
- situazione del traffico;
- informazioni relative all'emergenza;
- stato generale del sistema.

Una delle componenti più significative è la visualizzazione geografica dell'ambulanza.

L'operatore può quindi osservare il veicolo mentre si avvicina progressivamente all'intersezione.

Questo rende il comportamento del sistema immediatamente comprensibile anche durante una dimostrazione.

18. Worldmap e visualizzazione geografica

Nel flow Node-RED è presente la componente Worldmap.

Questa viene utilizzata per rappresentare graficamente la posizione del veicolo.

Il sistema riceve le coordinate dell'ambulanza e le utilizza per aggiornare la posizione visualizzata.

Il risultato è una rappresentazione dinamica del percorso:

```
graph TD; A[Posizione 1] --> B[Posizione 2]; B --> C[Posizione 3]; C --> D["..."]; D --> E[Intersezione];
```

La mappa costituisce quindi una vera interfaccia di monitoraggio del dispositivo Edge mobile.

19. Grafana

Grafana viene utilizzato principalmente come strumento di visualizzazione e analisi dei dati storici.

La dashboard realizzata contiene diversi pannelli.

Tra quelli mostrati nella configurazione attuale figurano:

Distanza ambulanza incrocio

Visualizza l'evoluzione temporale della distanza dell'ambulanza dall'intersezione.

Il grafico permette di osservare l'avvicinamento del veicolo e i diversi eventi registrati.

Andamento auto

Visualizza l'andamento del numero di auto/coda rilevato nel tempo.

Questo consente di osservare l'evoluzione del traffico ordinario.

Traffico attuale all'incrocio

Fornisce una rappresentazione immediata dello stato del traffico.

Attivazioni Green Wave

È previsto inoltre un pannello dedicato alle attivazioni della Green Wave.

Durante lo sviluppo è emersa una necessità di adeguamento del formato temporale dei dati per permettere una corretta visualizzazione temporale del pannello.

20. Differenza tra dashboard operativa e Grafana

Le due interfacce non hanno necessariamente lo stesso obiettivo.

La dashboard operativa è orientata principalmente al **monitoraggio real-time**:

Dove si trova l'ambulanza?
Quanto dista?
Qual è lo stato del semaforo?
Cosa sta succedendo adesso?

Grafana è invece particolarmente utile per l'analisi:

Cosa è successo?
Quando?
Con quale andamento?
Quante attivazioni ci sono state?
Come è variata la coda?
Come è variata la distanza?

La combinazione dei due strumenti permette quindi di avere sia una vista operativa sia una vista analitica.

21. Logging degli eventi

Un sistema di gestione delle emergenze non deve limitarsi a reagire agli eventi.

È importante anche poterli registrare.

Per questo motivo il progetto comprende una componente di logging.

Il flusso Node-RED include il nodo:

Formattazione csv

che prepara i dati dell'evento in formato CSV.

Il file viene quindi memorizzato come log dell'emergenza.

Nel progetto è presente un file denominato:

```
GreenWave-EMSlog_emergence.csv
```

Il log può contenere le informazioni raccolte durante l'evento e costituisce una base per:

- analisi successiva;
- debugging;
- valutazione delle prestazioni;
- generazione del report AI;
- archiviazione degli eventi.

22. Download del CSV da parte dell'utente

Durante l'evoluzione del progetto è stata aggiunta una funzionalità ulteriore rispetto alla semplice memorizzazione del file.

L'utente può infatti richiedere il **download del CSV direttamente sul proprio dispositivo**.

Questa caratteristica migliora l'usabilità del sistema perché il log non rimane solamente un file gestito internamente dal backend.

L'operatore può quindi esportare i dati relativi agli eventi e utilizzarli successivamente per:

- analisi offline;
- archiviazione;
- presentazione;
- elaborazione tramite altri strumenti;
- documentazione delle emergenze.

Questa funzionalità rende il logging più vicino a un vero strumento operativo e non solamente a un meccanismo di debug.

23. Endpoint HTTP

Nel sistema sono inoltre presenti interfacce HTTP utilizzate per esporre informazioni elaborate dal backend.

Nel flow sono visibili endpoint relativi a:

```
stato_semaforo
```

telemetria_ambulanza

coda_traffico

Questa componente permette di separare ulteriormente il livello di elaborazione dal livello di visualizzazione.

Node-RED può quindi ricevere ed elaborare i dati tramite MQTT e successivamente renderli disponibili attraverso interfacce HTTP utilizzabili dai componenti di visualizzazione.

24. Integrazione con il Large Language Model

Una delle caratteristiche più avanzate del progetto è l'integrazione tra IoT e Large Language Model.

L'obiettivo non è utilizzare l'LLM per controllare direttamente il semaforo.

La sicurezza e il controllo deterministico dell'intersezione rimangono affidati alla logica tradizionale.

Il modello linguistico viene invece utilizzato nella fase successiva all'emergenza.

La sua funzione è trasformare dati tecnici e telemetrici in un **report operativo comprensibile da un operatore umano**.

25. Ollama

Per l'integrazione AI viene utilizzato **Ollama**, con un modello linguistico locale.

La scelta di un modello locale presenta alcuni vantaggi:

- non richiede necessariamente l'invio dei dati a un servizio cloud esterno;
- permette di mantenere il processing sul sistema locale;
- rende il prototipo indipendente da servizi cloud commerciali;
- permette di sperimentare l'integrazione tra IoT e LLM in un ambiente controllato.

Nel progetto viene utilizzato un modello della famiglia indicata nella fase di progettazione, come ad esempio **Llama 3.2**.

26. Pipeline LLM

La pipeline Node-RED relativa all'intelligenza artificiale comprende diversi passaggi.

Nel flow sono visibili componenti come:

Generazione Prompt LLM



HTTP request



Estrazione Output IA

Il comportamento generale è:

Evento di emergenza



Raccolta telemetria



Aggregazione informazioni



Preparazione JSON



Generazione prompt



HTTP request verso Ollama



LLM



Risposta testuale



Estrazione output



Report operativo

In questo modo il modello non riceve semplicemente un singolo valore, ma può ricevere il contesto dell'evento.

27. Report operativo generato dall'AI

L'idea del report è trasformare informazioni come:

- distanza dell'ambulanza;
- attivazione della Green Wave;
- durata dell'evento;
- stato del semaforo;
- eventuale rilevamento di ostacoli/pedoni;
- comportamento della barriera;
- condizioni del traffico;

in una descrizione leggibile.

Un report può, ad esempio, sintetizzare:

- se l'emergenza è stata gestita correttamente;
- se sono stati rilevati impedimenti;
- se l'attivazione della Green Wave ha subito ritardi;
- quali condizioni particolari sono state osservate.

L'LLM diventa quindi uno strumento di **analisi post-evento**.

È importante sottolineare che il modello non sostituisce la logica deterministica di sicurezza.

Il suo compito è interpretativo e descrittivo.

28. Perché l'LLM è utile in questo progetto

Un sistema IoT può produrre moltissimi dati, ma i dati grezzi non sono sempre immediatamente comprensibili.

Per esempio, un log potrebbe contenere:

```
timestamp
distance
traffic_light
pedestrian
barrier
green_wave
duration
```

Un operatore dovrebbe interpretare manualmente questi dati.

L'LLM permette invece di trasformare una sequenza di valori in una sintesi:

```
L'ambulanza ha raggiunto l'intersezione durante una condizione
di emergenza. La Green Wave è stata attivata durante
l'avvicinamento. La procedura di attraversamento ha subito
un ritardo dovuto al rilevamento di un ostacolo.
```

Il valore dell'AI è quindi soprattutto nella **human-readable interpretation** dei dati IoT.

29. AI-assisted analysis, non controllo autonomo

Dal punto di vista tecnico è più corretto descrivere questa parte come:

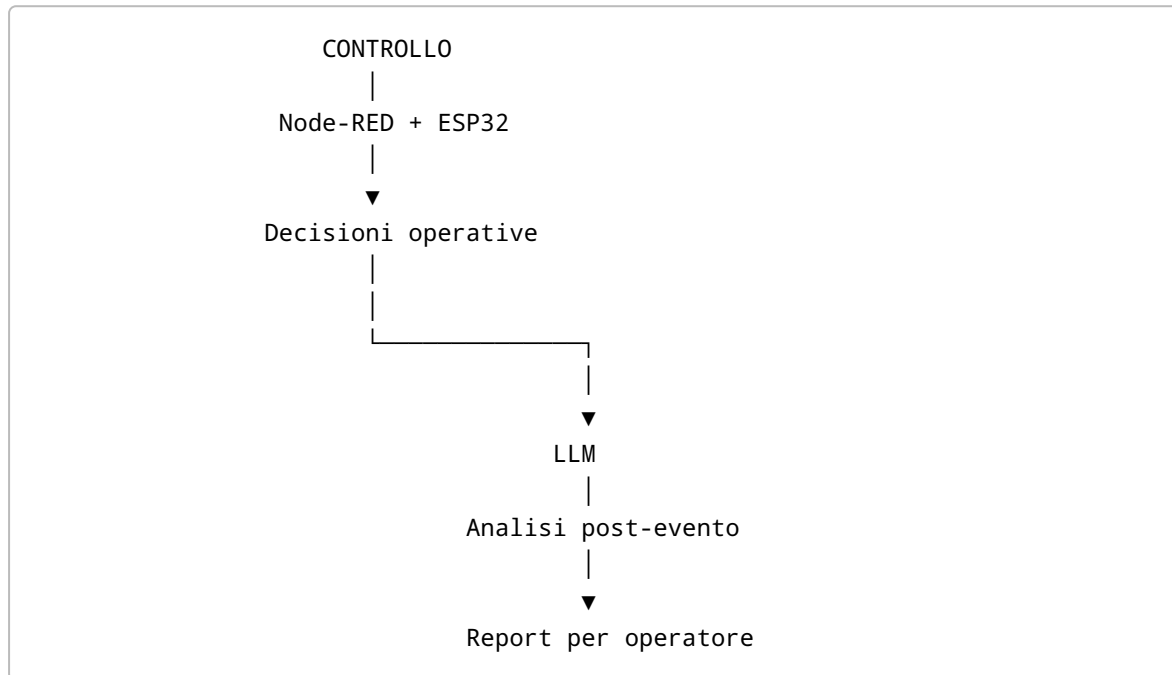
AI-assisted operational analysis

o:

IoT + LLM integration

piuttosto che considerarla necessariamente un sistema completamente autonomo.

Il sistema mantiene infatti una separazione importante:



Questa separazione aumenta l'affidabilità dell'architettura.

30. Gestione del traffico ordinario

Il progetto non considera esclusivamente l'ambulanza.

L'intersezione continua infatti a monitorare anche il traffico ordinario.

La coda viene raccolta tramite il sistema IoT e inviata al backend.

Node-RED mantiene una memoria della coda e rende disponibili i dati per la visualizzazione e l'analisi.

Questo permette di osservare il compromesso tra:

- priorità al veicolo di emergenza;
- gestione del traffico ordinario.

In un sistema reale questo sarebbe particolarmente importante, perché una Green Wave troppo lunga potrebbe causare congestione nelle altre direzioni.

31. Sviluppo parallelo del progetto

Il progetto è stato sviluppato da due persone attraverso una divisione delle responsabilità tra **Edge** e **Cloud/Backend**.

Questa scelta è stata adottata per permettere lo sviluppo parallelo delle componenti.

32. Sviluppo della componente Cloud/Backend

La componente Backend è stata sviluppata principalmente da Alessio.

La prima fase ha riguardato la configurazione dell'ambiente Node-RED e la creazione dei primi flussi MQTT.

Inizialmente sono stati utilizzati messaggi simulati tramite nodi Inject, in modo da poter sviluppare la logica anche prima del completamento degli Edge Device.

Successivamente è stata sviluppata la logica centrale:

- ricezione MQTT;
- interpretazione dei messaggi;
- calcolo della distanza;
- gestione della Green Wave;
- gestione dello stato dell'intersezione;
- gestione della coda;
- logging;
- comunicazione HTTP;
- integrazione Grafana;
- integrazione LLM.

Questo approccio ha permesso di sviluppare il backend indipendentemente dall'hardware.

33. Sviluppo della componente Edge

La componente Edge è stata sviluppata principalmente da Giorgio.

Il primo Edge Device è stato implementato tramite Node.js.

Il simulatore è stato progettato come un processo asincrono in grado di:

- generare coordinate;
- modificarle progressivamente;
- simulare l'avvicinamento all'intersezione;
- pubblicare i dati via MQTT.

Successivamente è stata sviluppata la seconda componente Edge tramite Wokwi.

34. Sviluppo dell'ESP32

La realizzazione dell'intersezione ha seguito una progressione logica.

Prima è stata costruita la struttura hardware virtuale:

```
ESP32
├─ LED rosso
├─ LED giallo
├─ LED verde
├─ HC-SR04
└─ Servo
```

Successivamente è stata implementata la logica locale degli attuatori.

Infine sono state aggiunte:

- connessione Wi-Fi;
- connessione MQTT;
- ricezione dei comandi;
- pubblicazione dello stato;
- lettura del sensore;
- Safety Logic.

Questo approccio ha permesso di testare separatamente la parte hardware e la parte di comunicazione.

35. Fase di integrazione

Dopo lo sviluppo parallelo delle due componenti è stata effettuata l'integrazione.

L'ambiente completo comprende:

```
Node.js
+
ESP32/Wokwi
+
MQTT Broker
+
Node-RED
+
Grafana
+
Dashboard
```

+
Ollama

Durante questa fase è stato necessario verificare soprattutto la coerenza dei dati.

Tra i possibili problemi di integrazione rientrano:

- topic MQTT non coincidenti;
- JSON con struttura differente;
- nomi dei campi differenti;
- valori booleani/stringhe interpretati diversamente;
- gestione dello stato;
- sincronizzazione degli eventi;
- gestione del reset.

Questa fase è stata fondamentale perché ha trasformato componenti sviluppate separatamente in un unico sistema.

36. Flusso completo di un'emergenza

Il comportamento completo del sistema può essere descritto attraverso il seguente scenario.

Fase 1 – Avvio dell'emergenza

Il simulatore Node.js porta l'ambulanza nello stato di emergenza.

La telemetria viene pubblicata tramite MQTT.

Fase 2 – Avvicinamento

Le coordinate dell'ambulanza cambiano progressivamente.

Node-RED riceve i dati.

La dashboard aggiorna la posizione del veicolo.

La distanza dall'intersezione viene calcolata e visualizzata.

Fase 3 – Raggiungimento della soglia

Quando la distanza raggiunge il livello stabilito dalla logica, Node-RED riconosce la necessità di preparare l'intersezione.

Viene generato il comando di Green Wave.

Fase 4 – Invio del comando

Il comando viene pubblicato tramite:

```
city/ems/trafficlight1/cmd
```

L'ESP32 riceve il messaggio.

Fase 5 – Safety Check

L'ESP32 verifica localmente le condizioni rilevate dal sensore HC-SR04.

Se non sono presenti condizioni di pericolo, procede con l'attuazione.

Se viene rilevato un ostacolo critico, la procedura viene temporaneamente inibita.

Fase 6 – Attuazione

Il sistema aggiorna:

- stato dei LED;
- stato del semaforo;
- posizione della barriera tramite servo.

L'intersezione viene predisposta al passaggio dell'ambulanza.

Fase 7 – Monitoraggio

Durante l'evento vengono raccolti dati relativi a:

- posizione;
- distanza;
- stato del semaforo;
- condizioni di sicurezza;
- traffico;
- Green Wave.

I dati vengono mostrati nella dashboard e registrati.

Fase 8 – Attraversamento

L'ambulanza attraversa l'intersezione.

Il sistema continua a monitorare l'evento.

Fase 9 – Fine emergenza

Quando l'evento termina, l'intersezione viene riportata allo stato normale.

Il sistema effettua il reset della situazione di emergenza.

Fase 10 – Logging

I dati raccolti vengono formattati e salvati nel file CSV dell'emergenza.

L'utente può inoltre scaricare il file direttamente sul proprio dispositivo.

Fase 11 – Analisi AI

I dati dell'evento vengono aggregati e utilizzati per costruire il prompt destinato al modello linguistico.

Node-RED effettua una richiesta HTTP verso Ollama.

Fase 12 – Report operativo

Il modello analizza i dati e produce un report in linguaggio naturale.

L'output viene estratto e reso disponibile al sistema.

In questo modo l'intero ciclo dell'emergenza viene trasformato da una semplice sequenza di messaggi IoT in un evento documentato e analizzabile.

37. Persistenza dei dati

La persistenza svolge un ruolo importante nel progetto.

I dati non vengono utilizzati solamente nell'istante in cui vengono prodotti.

Vengono infatti utilizzati per:

- dashboard;
- Grafana;
- CSV;
- analisi successive;
- report LLM.

Questo introduce una separazione tra:

real-time processing

e

historical analysis.

Il primo è necessario per la Green Wave.

Il secondo è utile per valutare il comportamento del sistema dopo l'evento.

38. Separazione tra controllo e monitoraggio

Un aspetto architetturale importante è la separazione tra controllo e visualizzazione.

La dashboard non costituisce il cuore decisionale del sistema.

Il processo decisionale rimane nel backend e nell'Edge Device.

La dashboard visualizza invece lo stato risultante.

Quindi:

```
Sensori
  ↓
MQTT
  ↓
Node-RED
  ↓
Logic
  ↓
MQTT
  ↓
ESP32
  ↓
Actuators
```

mentre parallelamente:

```
MQTT / Node-RED
  ↓
Dashboard / Grafana
  ↓
Human operator
```

Questo evita di confondere l'interfaccia utente con il sistema di controllo.

39. Vantaggi dell'architettura ibrida

La scelta di un'architettura ibrida permette di combinare diversi vantaggi.

Simulazione

Wokwi permette di sviluppare e testare l'ESP32 senza hardware fisico.

Modularità

MQTT permette di sostituire o modificare singoli dispositivi senza riprogettare l'intero sistema.

Elaborazione centralizzata

Node-RED permette di implementare rapidamente logiche complesse.

Sicurezza locale

L'ESP32 può mantenere decisioni di sicurezza anche quando riceve comandi dal backend.

Visualizzazione

Dashboard e Grafana permettono di osservare sia lo stato corrente sia lo storico.

AI locale

Ollama consente di aggiungere capacità linguistiche senza rendere necessariamente il sistema dipendente da un servizio cloud esterno.

40. Perché non utilizzare direttamente l'LLM per la Green Wave

Una scelta progettuale importante è mantenere l'LLM fuori dal percorso critico della sicurezza.

La Green Wave viene determinata attraverso una logica deterministica basata su parametri misurabili, come la distanza.

Questo è preferibile rispetto a una decisione affidata a un modello probabilistico.

Il modello linguistico viene quindi utilizzato dopo l'evento, quando l'obiettivo è interpretare i dati.

In questo modo:

CONTROLLO CRITICO
→ logica deterministica

ANALISI E REPORTING
→ LLM

La separazione è più adatta a un sistema di emergenza.

41. Evoluzione del progetto rispetto alla proposta iniziale

La proposta iniziale definiva già i principali elementi del progetto:

- ambulanza simulata;
- ESP32;
- Wokwi;
- MQTT;
- Node-RED;
- dashboard;
- CSV;
- LLM.

Durante lo sviluppo alcune funzionalità sono state ulteriormente concretizzate o ampliate.

In particolare, il sistema realizzato comprende anche:

- monitoraggio della coda del traffico;
- gestione dello stato dell'intersezione;
- reset dell'incrocio;
- endpoint HTTP;
- visualizzazione geografica;
- dashboard operativa;
- Grafana per l'analisi;
- download del CSV da parte dell'utente;
- pipeline strutturata per l'estrazione dell'output LLM.

Queste caratteristiche possono essere considerate un'evoluzione naturale della proposta iniziale.

42. Oggetti e tecnologie utilizzati

Hardware / Hardware simulato

ESP32

Microcontrollore utilizzato come Edge Device dell'intersezione.

HC-SR04

Sensore a ultrasuoni utilizzato per il rilevamento locale di ostacoli.

LED rosso

Rappresentazione del segnale rosso.

LED giallo

Rappresentazione del segnale giallo.

LED verde

Rappresentazione del segnale verde.

Servomotore

Simulazione della barriera di sicurezza.

Wokwi

Ambiente di simulazione utilizzato per sviluppare e testare l'intersezione basata su ESP32.

43. Software e servizi

Node.js

Utilizzato per simulare il dispositivo Edge mobile, cioè l'ambulanza.

JavaScript

Utilizzato per la logica del simulatore e per le Function Node di Node-RED.

C++

Utilizzato per il firmware dell'ESP32.

MQTT

Protocollo di comunicazione tra dispositivi e backend.

MQTT Broker

Intermediario della comunicazione publish/subscribe.

Node-RED

Backend e stream processor centrale.

Grafana

Sistema di visualizzazione e analisi storica.

Worldmap

Sistema di visualizzazione geografica della posizione dell'ambulanza.

Ollama

Runtime locale per il Large Language Model.

LLM

Utilizzato per la generazione automatica del report operativo post-emergenza.

44. Struttura logica del sistema

L'intero progetto può essere riassunto in cinque livelli.

Livello 1 – Perception

Comprende:

- GPS simulato;
 - HC-SR04;
 - stato dell'ambulanza;
 - rilevamento della situazione del traffico.
-

Livello 2 – Transport

Comprende:

- MQTT;

- broker;
 - topic dedicati.
-

Livello 3 – Processing

Comprende:

- Node-RED;
 - calcolo della distanza;
 - Green Wave;
 - gestione della coda;
 - stato del semaforo;
 - logging;
 - preparazione dei dati.
-

Livello 4 – Application

Comprende:

- dashboard operativa;
 - Worldmap;
 - Grafana;
 - visualizzazione dello stato;
 - esportazione CSV.
-

Livello 5 – AI

Comprende:

- aggregazione dell'evento;
 - generazione del prompt;
 - Ollama;
 - LLM;
 - estrazione della risposta;
 - report operativo.
-

45. Aspetti di sicurezza

La sicurezza è stata considerata soprattutto attraverso la Safety Logic locale.

La scelta di non affidare completamente l'attuazione al backend è importante.

Un sistema reale non dovrebbe assumere che:

comando ricevuto = comando sempre sicuro da eseguire

L'ESP32 verifica invece le condizioni locali.

Questo introduce una forma di **defense in depth**, nella quale il backend decide l'obiettivo generale mentre il dispositivo Edge conserva il controllo delle condizioni fisiche immediate.

46. Affidabilità e simulazione

Il progetto utilizza una combinazione di simulazione e software locale.

Questo permette di effettuare dimostrazioni senza dipendere dalla disponibilità di hardware fisico reale.

In particolare:

- l'ambulanza è simulata;
- l'intersezione è simulata tramite Wokwi;
- il backend è eseguito tramite Node-RED;
- il broker gestisce la comunicazione;
- Grafana visualizza i dati;
- Ollama fornisce l'elaborazione linguistica.

L'architettura consente quindi di riprodurre un caso d'uso IoT completo in un ambiente controllato.

47. Possibili scenari di test

Il sistema può essere testato attraverso diversi scenari.

Scenario normale

L'ambulanza non è in emergenza.

Il semaforo continua a funzionare normalmente.

Scenario di avvicinamento

L'ambulanza entra nello stato di emergenza e si avvicina all'incrocio.

Il sistema visualizza il movimento e la diminuzione della distanza.

Scenario Green Wave

La distanza raggiunge la soglia.

Node-RED genera il comando di Green Wave.

L'ESP32 riceve il comando.

Scenario con ostacolo

Il comando viene ricevuto ma il sensore rileva un ostacolo.

La Safety Logic locale impedisce o ritarda l'attuazione.

Scenario senza ostacolo

Il sensore non rileva condizioni critiche.

L'intersezione procede con la procedura di emergenza.

Scenario fine emergenza

L'ambulanza supera l'intersezione.

Il sistema esegue il reset e torna alla condizione normale.

Scenario post-emergenza

I dati vengono salvati nel CSV.

Il file può essere scaricato.

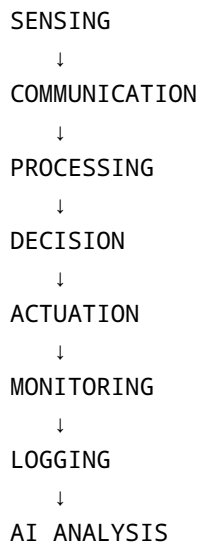
Il dataset viene passato alla pipeline LLM.

Il sistema genera il report operativo.

48. Risultato finale

Il risultato del progetto è un prototipo completo di sistema IoT per la gestione intelligente di un'intersezione stradale in presenza di un veicolo di emergenza.

Il sistema realizza un ciclo completo:



Questo aspetto è particolarmente importante perché il progetto non si limita alla realizzazione di un singolo dispositivo o di una singola dashboard.

Sono state integrate diverse tecnologie in una pipeline unica.

49. Valutazione dell'architettura

Dal punto di vista IoT, il progetto presenta diversi elementi significativi.

Distribuzione

Sono presenti più dispositivi e componenti distribuiti.

Comunicazione

MQTT permette la comunicazione asincrona.

Edge Computing

L'ESP32 implementa una logica locale di sicurezza.

Processing

Node-RED centralizza l'elaborazione.

Visualization

Dashboard e Grafana rendono i dati accessibili all'operatore.

Persistence

I dati vengono conservati tramite logging e database/sistemi utilizzati dalla visualizzazione.

AI

L'LLM aggiunge un livello di interpretazione dei dati.

La combinazione di questi elementi rende il progetto rappresentativo di un'architettura IoT completa.

50. Conclusioni

GreenWave EMS è stato sviluppato come un sistema IoT ibrido destinato a simulare la gestione intelligente di un'intersezione stradale durante il passaggio di un veicolo di emergenza.

Il progetto parte da un problema reale delle Smart Cities: ridurre il tempo necessario a un'ambulanza per attraversare intersezioni congestionate senza compromettere la sicurezza.

La soluzione realizzata combina due Edge Device distinti.

Il primo è un simulatore Node.js che rappresenta l'ambulanza e produce la telemetria relativa alla posizione e allo stato dell'emergenza.

Il secondo è un'intersezione intelligente basata su ESP32 e simulata tramite Wokwi, dotata di sensore HC-SR04, LED e servomotore.

La comunicazione tra i componenti avviene tramite MQTT secondo il modello publish/subscribe.

Node-RED rappresenta il centro di elaborazione del sistema. Riceve la telemetria, calcola la distanza dell'ambulanza dall'intersezione, gestisce la logica della Green Wave, raccoglie informazioni sul traffico, mantiene lo stato del sistema e coordina le comunicazioni con l'intersezione.

Una caratteristica particolarmente importante è la separazione tra orchestrazione centrale e sicurezza locale. Node-RED può richiedere l'attivazione della Green Wave, ma l'ESP32 mantiene una propria Safety Logic basata sul sensore locale. In questo modo la decisione relativa all'attuazione fisica non viene affidata esclusivamente al backend.

Il sistema comprende inoltre un livello di monitoraggio costituito da una dashboard operativa e da Grafana. La dashboard permette di seguire l'ambulanza in movimento, visualizzare la sua posizione, controllare la distanza dall'incrocio e osservare lo stato dell'intersezione. Grafana viene utilizzato per analizzare dati storici relativi alla distanza, al traffico e alle attivazioni del sistema.

La gestione dei dati viene completata da un sistema di logging CSV. Gli eventi possono essere registrati e il file risultante può essere scaricato direttamente dall'utente sul proprio dispositivo, rendendo i dati disponibili per analisi e archiviazione successive.

Infine, il progetto integra un Large Language Model locale tramite Ollama. Al termine dell'evento, i dati raccolti vengono utilizzati per costruire un contesto strutturato che viene inviato al modello. L'LLM produce quindi un report operativo in linguaggio naturale, trasformando una serie di dati IoT in una sintesi comprensibile all'operatore.

La componente AI non viene utilizzata per prendere decisioni critiche sul controllo del traffico. Il percorso decisionale e la Safety Logic rimangono deterministici e affidati a Node-RED e all'ESP32. L'intelligenza artificiale viene invece impiegata per l'interpretazione e la documentazione dell'evento.

Il risultato finale è quindi un sistema che integra:

Edge Computing + MQTT + Stream Processing + Smart Actuation + Real-Time Monitoring + Historical Analysis + Data Logging + Local LLM.

L'aspetto più significativo del progetto è proprio l'integrazione di questi livelli. GreenWave EMS non rappresenta solamente un semaforo controllato tramite MQTT, ma un'intera pipeline IoT nella quale un evento fisico simulato viene acquisito, trasmesso, elaborato, trasformato in un'azione, monitorato, registrato e infine analizzato automaticamente.

L'architettura ottenuta costituisce quindi un prototipo di Smart City nel quale l'infrastruttura stradale diventa parte attiva del sistema di gestione delle emergenze.

Parte di Alessio – Componente Backend e simulazione del traffico

Questa sezione integra il report descrivendo, in termini esclusivamente concettuali, le componenti sviluppate nella parte Backend/Cloud e i due simulatori JavaScript utilizzati per rappresentare il comportamento dell'ambulanza e del traffico. Non vengono riportati dettagli di codice.

1. Ruolo della componente Backend

La componente Backend costituisce il livello centrale di elaborazione del sistema. Il suo compito è ricevere i dati provenienti dai dispositivi e dai simulatori, interpretarli, produrre informazioni derivate e coordinare il comportamento dell'intersezione. Nel progetto questa responsabilità è stata principalmente sviluppata da Alessio.

Lo sviluppo è iniziato predisponendo l'ambiente Node-RED e i primi flussi MQTT. In una fase iniziale sono stati utilizzati messaggi simulati, così da poter sviluppare e verificare la logica anche prima che tutti gli Edge Device fossero completati. Successivamente la componente Backend è stata estesa con il calcolo della distanza, la gestione della Green Wave, lo stato dell'intersezione, il monitoraggio della coda, il logging, le interfacce HTTP, Grafana e l'integrazione LLM.

2. Gestione della telemetria dell'ambulanza

Il simulatore dell'ambulanza rappresenta il dispositivo Edge mobile. L'ambulanza viene fatta muovere virtualmente lungo un percorso costituito da una fase di avvicinamento all'intersezione e una successiva fase di allontanamento. A intervalli regolari il simulatore aggiorna la posizione e invia la telemetria tramite MQTT.

La telemetria contiene concettualmente la posizione geografica, la distanza dall'intersezione e lo stato della sirena. In questo modo Node-RED riceve una sequenza temporale di informazioni che permette di ricostruire l'avvicinamento del mezzo e di determinare quando è necessario preparare l'incrocio.

La distanza viene calcolata considerando le coordinate geografiche dell'ambulanza e quelle dell'intersezione. Il calcolo tiene quindi conto della natura geografica delle coordinate, fornendo una distanza espressa in metri. Questo valore viene poi utilizzato dal Backend come parametro decisionale per la Green Wave.

3. Logica della Green Wave

La Green Wave è una conseguenza automatica della telemetria ricevuta. Quando l'ambulanza è ancora sufficientemente lontana, l'intersezione continua a funzionare normalmente. Quando la distanza scende sotto la soglia prevista, il Backend riconosce che l'evento di emergenza richiede la preparazione dell'intersezione.

Il Backend invia quindi un comando MQTT all'intersezione. L'ESP32 riceve il comando, ma non lo esegue in modo cieco: prima effettua il proprio controllo locale di sicurezza. Questa separazione tra decisione centrale e verifica locale è fondamentale perché il Backend stabilisce l'obiettivo operativo, mentre l'Edge Device mantiene il controllo sulle condizioni fisiche immediate.

4. Simulazione del traffico

Una seconda componente JavaScript simula una telecamera o un sensore di traffico associato all'intersezione. Il simulatore mantiene una stima del numero di veicoli in coda e la aggiorna periodicamente, producendo variazioni differenti a seconda che l'intersezione si trovi in condizione normale oppure in fase di Green Wave.

In condizioni normali la coda può aumentare o diminuire in modo variabile, simulando l'andamento del traffico ordinario. Quando la Green Wave è attiva, il modello rappresenta invece un deflusso della coda: il numero di

veicoli tende progressivamente a diminuire. Il valore viene mantenuto entro limiti prestabiliti per rappresentare una situazione fisicamente plausibile.

Oltre al numero di veicoli, il simulatore associa alla coda un livello qualitativo di congestione, distinguendo tra traffico basso, medio e alto. Queste informazioni vengono pubblicate tramite MQTT e possono essere utilizzate da Node-RED per il monitoraggio, la visualizzazione e l'analisi storica.

5. Interazione tra traffico e Green Wave

Il simulatore del traffico non è indipendente dall'evento di emergenza. Esso ascolta il topic MQTT utilizzato per i comandi dell'intersezione e riconosce l'attivazione della Green Wave. In questo modo la simulazione del traffico reagisce allo stesso evento che modifica il comportamento del semaforo.

Questo permette di rappresentare concettualmente il compromesso tipico di un sistema di emergenza: l'ambulanza riceve priorità, ma il sistema continua a osservare cosa accade al traffico ordinario. Una Green Wave troppo prolungata potrebbe infatti favorire il mezzo di emergenza ma aumentare la congestione nelle altre direzioni.

6. MQTT come collegamento tra simulatori e Backend

I simulatori non comunicano direttamente con Node-RED. Utilizzano MQTT e un broker come intermediario. La telemetria dell'ambulanza viene pubblicata sul canale dedicato alla posizione del veicolo, mentre la simulazione del traffico pubblica i dati relativi alla coda sul relativo topic. I comandi destinati all'intersezione vengono invece inviati sul topic di controllo.

Questa organizzazione realizza un modello publish/subscribe nel quale produttori e consumatori dei dati sono disaccoppiati. Di conseguenza, il Backend può elaborare i dati senza dover conoscere direttamente l'implementazione interna dei simulatori.

7. Flusso complessivo della mia componente

- Il simulatore dell'ambulanza aggiorna progressivamente la posizione e comunica la telemetria.
- Il Backend riceve la telemetria e ricava la distanza dall'intersezione.
- Il valore della distanza viene confrontato con la soglia che determina l'intervento.
- Quando necessario, il Backend genera il comando di Green Wave e lo invia tramite MQTT.
- L'intersezione riceve il comando e applica la propria Safety Logic prima dell'attuazione.
- Parallelamente, il simulatore del traffico continua a produrre dati sulla coda e reagisce allo stato di Green Wave.
- I dati vengono raccolti dal Backend per dashboard, Grafana, logging e analisi post-evento.

8. Aspetti concettuali importanti per l'esame

Separazione delle responsabilità: il Backend coordina e prende le decisioni di orchestrazione, mentre l'ESP32 mantiene la verifica locale delle condizioni di sicurezza.

Elaborazione real-time: la distanza dell'ambulanza e lo stato dell'emergenza devono essere elaborati durante l'evento per attivare tempestivamente la Green Wave.

Monitoraggio storico: i dati sulla distanza, sulla coda e sulle attivazioni vengono conservati per poter analizzare il comportamento del sistema dopo l'emergenza.

Disaccoppiamento tramite MQTT: i componenti comunicano attraverso un broker e topic distinti, rendendo l'architettura modulare e facilmente estendibile.

Controllo deterministico: la decisione critica relativa alla Green Wave non viene affidata all'LLM. L'intelligenza artificiale viene utilizzata successivamente per interpretare e sintetizzare i dati dell'evento.

9. Collegamento con il resto del progetto

La parte Backend e i simulatori completano il ciclo IoT del progetto: sensing e simulazione dei dati, comunicazione tramite MQTT, elaborazione centralizzata in Node-RED, decisione, attuazione sull'Edge Device, monitoraggio, logging e analisi post-evento. La componente sviluppata da Alessio costituisce quindi il collegamento logico tra la telemetria prodotta dai dispositivi e le funzionalità applicative del sistema, mantenendo separato il percorso critico di controllo dalla fase di analisi.

Parte aggiuntiva – Semaforo intelligente dell'incrocio

Questa sezione descrive concettualmente il componente Edge che rappresenta l'intersezione semaforica. La spiegazione è volutamente priva di codice e si concentra sul funzionamento, sulla comunicazione con il Backend e sulla logica di sicurezza locale.

10. Struttura del semaforo intelligente

L'intersezione è realizzata tramite un ESP32, simulato nell'ambiente Wokwi. Il microcontrollore coordina gli elementi fisici dell'incrocio: tre LED rappresentano rispettivamente il rosso, il giallo e il verde, un sensore a ultrasuoni rileva la presenza di un pedone o di un ostacolo nell'area controllata e un servomotore rappresenta una barriera o un varco di sicurezza.

Il componente Edge dispone quindi sia di elementi di attuazione sia di un elemento di rilevamento. Questa caratteristica è importante perché l'ESP32 non si limita a ricevere ordini dal Backend, ma può valutare autonomamente le condizioni locali prima di modificare lo stato dell'incrocio.

11. Comunicazione tra ESP32 e Backend

L'ESP32 comunica tramite Wi-Fi con il broker MQTT. Riceve dal Backend i comandi relativi allo stato dell'intersezione attraverso un topic di controllo e pubblica periodicamente il proprio stato attraverso un topic dedicato. Il Backend può quindi sapere quale stato del semaforo è effettivamente presente sull'Edge Device.

Il modello è quindi bidirezionale: Node-RED invia una richiesta di comportamento, mentre l'ESP32 comunica il risultato e le condizioni rilevate localmente. Questo permette di distinguere il comando richiesto dallo stato effettivamente attuato.

12. Rilevamento locale del pedone

Il sensore HC-SR04 viene utilizzato per verificare la presenza di un ostacolo o di un pedone nella zona di attraversamento. L'ESP32 misura la distanza dell'oggetto rilevato e considera significativa la presenza di un pedone quando la distanza rientra nella soglia prevista dal progetto.

Il controllo viene effettuato localmente. Di conseguenza, il Backend non deve necessariamente conoscere in anticipo la situazione fisica dell'attraversamento per impedire un'attuazione non sicura: è l'Edge Device stesso a poter bloccare la procedura quando rileva una condizione critica.

13. Funzionamento normale dell'intersezione

In assenza di una richiesta di Green Wave, l'intersezione rimane nello stato normale previsto dal progetto. Nel comportamento implementato questo corrisponde allo stato rosso, con la barriera mantenuta nella posizione di sicurezza. L'ESP32 continua comunque a monitorare il sensore e la connessione MQTT.

Questo stato costituisce anche il comportamento di fallback: se non è presente un comando di Green Wave valido, il sistema non mantiene accidentalmente lo stato di emergenza.

14. Attivazione della Green Wave sull'Edge

Quando Node-RED riconosce che l'ambulanza si trova nella zona di intervento, invia il comando di Green Wave tramite MQTT. L'ESP32 riceve il comando e verifica contemporaneamente la condizione del sensore locale.

Se il comando richiede la Green Wave e non viene rilevato alcun pedone o ostacolo, l'ESP32 autorizza l'attuazione: il semaforo passa al verde e il servomotore porta la barriera nella posizione associata al passaggio dell'ambulanza. In questo modo il comando del Backend viene trasformato in un comportamento

fisico dell'intersezione.

15. Safety Logic: priorità alla sicurezza locale

La situazione più importante dal punto di vista concettuale è quella in cui arriva un comando di Green Wave ma il sensore rileva un pedone. In questo caso l'ESP32 non esegue semplicemente il comando ricevuto.

La procedura di emergenza viene localmente inibita: il verde non viene concesso e l'intersezione mantiene uno stato di blocco associato alla presenza del pedone, con barriera chiusa. Questo realizza la cosiddetta Safety Logic e dimostra la separazione tra orchestrazione centrale e sicurezza locale.

Il principio fondamentale è quindi: **Node-RED decide quando preparare l'intersezione, ma l'ESP32 decide se le condizioni locali permettono effettivamente di attuare il comando.** Un problema di rete o una decisione del Backend non deve automaticamente tradursi in un comportamento fisico non sicuro.

16. Monitoraggio dello stato del semaforo

L'ESP32 non si limita a comandare i LED, ma mantiene una rappresentazione dello stato effettivo dell'intersezione. Lo stato può distinguere almeno tra funzionamento normale, Green Wave autorizzata e Green Wave bloccata dalla presenza di un pedone.

Queste informazioni vengono periodicamente pubblicate verso Node-RED insieme allo stato di rilevamento del pedone e all'informazione di funzionamento del dispositivo. Il Backend può così utilizzare i dati per dashboard, Grafana e logging.

17. Temporizzazione non bloccante

Un ulteriore aspetto progettuale è l'uso di una gestione temporale che consente all'ESP32 di pubblicare periodicamente lo stato senza bloccare continuamente l'esecuzione del resto della logica. Questo permette di mantenere contemporaneamente attive la comunicazione MQTT, la lettura del sensore e la gestione dello stato del semaforo.

18. Flusso completo dell'incrocio

- L'ESP32 si collega alla rete Wi-Fi e al broker MQTT.
- L'ESP32 si iscrive al canale attraverso cui Node-RED invia i comandi.
- Il dispositivo monitora continuamente la presenza di pedoni o ostacoli tramite il sensore.
- In condizioni normali mantiene l'intersezione nello stato previsto dal sistema.
- Quando arriva una richiesta di Green Wave, l'ESP32 verifica prima le condizioni locali.
- Se il percorso è libero, attiva il verde e porta la barriera nella posizione prevista per il passaggio.
- Se viene rilevato un pedone, blocca localmente la Green Wave e mantiene una condizione sicura.
- L'ESP32 comunica periodicamente a Node-RED lo stato effettivo dell'intersezione e del sensore.

19. Concetti da ricordare per l'esame

Edge Computing: alcune decisioni vengono prese direttamente sul dispositivo vicino al componente fisico, senza dipendere completamente dal Backend.

Safety Logic: un comando proveniente dal Cloud/Backend non bypassa i controlli locali di sicurezza.

MQTT publish/subscribe: il Backend invia comandi tramite un topic, mentre l'ESP32 pubblica il proprio stato su un altro topic.

Separazione controllo/stato: il comando ricevuto rappresenta ciò che il Backend richiede; il messaggio di stato rappresenta ciò che l'Edge Device sta effettivamente facendo.

Integrazione con GreenWave EMS: la logica centrale calcola quando è necessario attivare la Green Wave, mentre il semaforo intelligente verifica localmente se l'attivazione è sicura.